RAILWAY DELAY FORECASTING · GRAPH NEURAL NETWORKS

Direct Multi-Horizon GATv2 for Train Delay Prediction

Replacing autoregressive rollout with direct multi-horizon decoding in an event-based, topology-aware graph attention framework — testing whether direct prediction curbs horizon-dependent error accumulation.

01 Motivation

Railway operations are a web of dependent events, not isolated observations. A primary delay at one train event can spread to neighboring events through shared infrastructure, dwell interactions, and headway constraints — producing secondary, knock-on delays that cascade through the network.

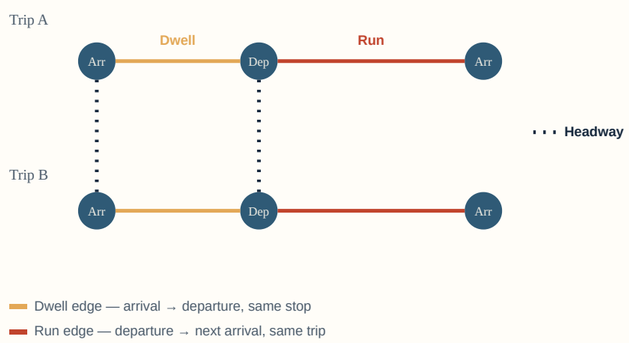
- Graph-based learning represents train events as nodes and operational dependencies as edges, letting message passing carry delay information between connected events.
- Prior work (Nguyen & Li) pairs an event-based GATv2 encoder with autoregressive rollout for multi-step forecasts — but recursively reusing predictions means an early error can compound at later horizons.
- **Research question:** is horizon degradation caused by the graph representation itself, or by the autoregressive decoding strategy layered on top of it?

02 Feature engineering

Dwell time	$D_i = t^{dep}_i - t^{arr}_i$
Running time	$R_i = t^{arr}_i - t^{dep}_{i-1}$
Headway	$H_i = t_i - t_{i-1}$

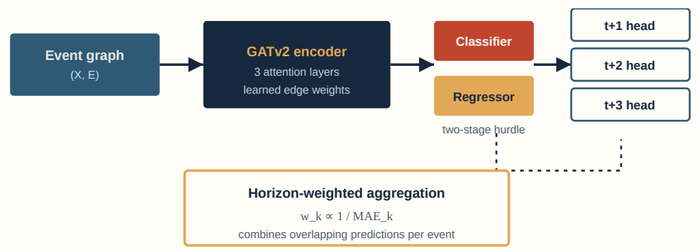
Trip context is captured with stop sequence, normalized trip progress, and stops remaining. Time of day is encoded with sine / cosine transforms plus a rush-hour indicator. Data comes from Subway Data NYC observed trips, matched to the official MTA static GTFS schedule by service day.

03 Graph construction



Every arrival and departure is a node (event-based graph). Dwell edges link an arrival to its departure at the same stop; Run edges link a departure to the next arrival on the same trip; Headway edges link consecutive trains sharing route, direction, stop, and event type.

04 Model architecture



A 3-layer GATv2 backbone encodes each event node, feeding a two-stage hurdle (classify delayed / not, then regress magnitude). The proposed Direct Multi-Horizon (DMH) heads predict t+1, t+2, t+3 simultaneously from the current graph state, instead of feeding predictions back in autoregressively. Overlapping horizon estimates for the same event are combined with validation-MAE-based weights.

05 Results

Table I — overall validation performance

Model	MAE	RMSE	Acc.	Prec.	Rec.	F1
Zero-delay	6.710	13.711	.517	.000	.000	.000
Persistence	0.729	1.893	.929	.938	.913	.926
Autoregressive GATv2	3.124	7.597	.922	.938	.899	.918
DMH GATv2 + agg.	**2.560**	**6.914**	**.946**	**.949**	**.938**	**.944**

Table II — horizon-wise MAE (min) and accuracy

Model	t+1	t+2	t+3
AR MAE	2.411	2.861	3.124
DMH MAE	**2.645**	**2.703**	**2.751**
AR Acc.	.957	.933	.922
DMH Acc.	**.957**	**.936**	**.926**

4.4 pts higher recall for DMH over autoregressive GATv2 (93.8% vs 89.9%), and DMH's MAE stays nearly flat across horizons (2.65 → 2.75 min) while the autoregressive model's error climbs from 2.41 to 3.12 min — the signature of compounding error.

06 Conclusion

Direct multi-horizon decoding preserves the topology-aware strengths of the event-based GATv2 encoder while measurably curbing horizon-dependent error growth. DMH is the stronger delay *detector* and the more stable regressor at longer horizons, even though the persistence baseline remains hard to beat on raw MAE — a reminder that subway delays are often locally stable, and that model value here lies in decision support rather than beating a naive baseline outright. Future work: sweep the horizon length K, and test on networks with more frequent sudden disruptions and congestion.

07 References

[1] Spanninger, Trivella, Büchel & Corman, "A review of train delay prediction approaches," *J. Rail Transport Planning & Management*, 22, 2022.

[2] Nguyen & Li, "Deep Learning to Identify the Spatio-Temporal Cascading Effects of Train Delays in a High-Density Network," Proc. GeoAI '25, ACM SIGSPATIAL, 2025.

[3] Kipf & Welling, "Semi-Supervised Classification with Graph Convolutional Networks," ICLR, 2017.

[4] "Subway Data NYC," subwaydata.nyc, accessed Sep. 2026.

[5] Brody, Alon & Yahav, "How Attentive Are Graph Attention Networks?" ICLR, 2022.